

TypeScript for Test Automation Handbook (2026 Edition)

Single Volume - From Zero to Confident, Built Entirely Around Playwright and Cypress Code

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Handbook	3
SECTION 1 — TypeScript Fundamentals for Test Automation	4
1.1 What TypeScript Actually Adds Over Plain JavaScript	4
1.2 Type Inference vs. Explicit Annotation	4
1.3 The Basic Types You'll See Every Day: string, number, boolean, arrays	5
1.4 any, unknown, and never — the Three Types That Confuse Everyone Coming From JavaScript	6
SECTION 2 — Objects, Interfaces & Type Aliases	7
2.1 Interfaces — Describing the Shape of Your Test Data	7
2.2 Optional Properties (?) and readonly	8
2.3 Type Aliases — and When They're Better Than an Interface	9
SECTION 3 — Functions in TypeScript	10
3.1 Typing Function Parameters and Return Values	10
3.2 Optional and Default Parameters	11
3.3 Typing Callback Functions (the Pattern Behind page.route() and cy.intercept())	12
SECTION 4 — Async/Await and Promises	13
4.1 Why Almost Everything in Playwright Is async	13
4.2 The Cypress Exception — Why cy. Commands Don't Use await	14
4.3 Typing a Function That Returns a Promise	14
SECTION 5 — Classes for Page Objects	15
5.1 Class Basics — Constructor, Properties, and Methods	15
5.2 Access Modifiers — private, public, and readonly	16
5.3 Composing Page Objects — a Class That Contains Other Classes	17
SECTION 6 — Union Types, Narrowing & Modern Syntax Shortcuts	19
6.1 Union Types — "This or That, Nothing Else"	19
6.2 Narrowing — Checking Which Option a Union Actually Is	20
6.3 Optional Chaining (?.) and Nullish Coalescing (??)	21
SECTION 7 — Generics (Made Simple, Tied Directly to Fixtures)	22
7.1 What Problem Generics Actually Solve — Before Any Syntax	22

7.2 Generics in Playwright's <code>test.extend<Fixtures>()</code> — Finally Explained	23
7.3 A Simpler, Everyday Generic You'll Actually Write Yourself: A Typed API Helper . . .	24
SECTION 8 — Utility Types You'll Actually Use	25
8.1 <code>Partial<T></code> — “All These Fields, But Optional”	25
8.2 <code>Pick<T, Keys></code> and <code>Omit<T, Keys></code> — Selecting or Excluding Specific Fields . . .	26
8.3 <code>Record<Keys, ValueType></code> — Typing an Object Used as a Lookup Map	27
SECTION 9 — Modules, Imports/Exports & Declaration Files	28
9.1 Named Exports vs. Default Exports	28
9.2 Declaring Custom Cypress Commands — the <code>.d.ts</code> File, Finally Explained	29
9.3 Importing Playwright's Own Types — Page, Locator, <code>APIRequestContext</code>	29
SECTION 10 — Common Errors You'll Hit This Week, and Exactly How to Fix Them	30
10.1 “Property ‘x’ does not exist on type ‘y’”	30
10.2 “Object is possibly ‘undefined’” or “possibly ‘null’”	31
10.3 “Argument of type ‘X’ is not assignable to parameter of type ‘Y’”	31
10.4 A Custom Cypress Command Shows a Type Error Even Though It Works Fine When You Run It	32
10.5 “This expression is not callable” or “Cannot invoke an object which is possibly undefined”	32
10.6 <code>tsconfig</code> strict Mode — What It Actually Turns On, in Plain English	33
SECTION 11 — Loops, Arrays, Maps & Sets	34
11.1 The Three Loops You'll Actually Use: <code>for...of</code> , <code>for...in</code> , and Plain <code>for</code>	34
11.2 <code>.forEach()</code> — the Array Method Version of a Loop	34
11.3 <code>.map()</code> and <code>.filter()</code> — Transforming and Narrowing Arrays Without a Loop . .	35
11.4 <code>.find()</code> , <code>.some()</code> , <code>.every()</code> , and <code>.includes()</code> — Asking Questions About an Array	36
11.5 Map — a Better “Object as a Lookup Table”	37
11.6 Set — a List With No Duplicates	38

How to Use This Handbook

You already know QA automation deeply - 14 years of it. What you don't have yet is TypeScript itself, and you've only ever touched it inside Playwright and Cypress code, not as a general programming language. This handbook is built around that exact situation: every single concept is explained once in plain English, then shown immediately in both a Playwright example and a Cypress example - never a generic "here's a class about animals" example that you'd have to mentally translate back to your actual work.

The ten sections build in order, each one assuming only what came before it. Sections 1-6 are things you will use in literally every test file you write. Sections 7-9 (generics, utility types, declaration files) are the parts that intimidate most people coming from a non-TypeScript background - they're explained slowly, because they're also the parts of the Playwright fixture system and Cypress custom commands that you've probably been copy-pasting without fully understanding until now. Section 10 is a troubleshooting guide for the exact red-squiggly-line errors you will hit this week.

Each entry has the same shape: what it is, why it actually matters for your work specifically, a plain-English explanation, a Playwright example, a Cypress example, the mistake beginners make with it, and a one-line practice tip.

Section 11 covers loops, array methods (map, filter, find), Map, and Set - the everyday data-wrangling tools you'll use in nearly every test that touches a list of roles, cohorts, or table rows.

SECTION 1 — TypeScript Fundamentals for Test Automation

1.1 What TypeScript Actually Adds Over Plain JavaScript

Level: Foundational | **You'll Use This:** Constantly, without thinking about it

Why It Matters For Your Work Every .ts test file you write is JavaScript plus a type-checking layer that runs *before* your test executes — catching mistakes like passing a number where a string is expected, at the moment you save the file, instead of discovering it when the test fails at runtime three steps into a Playwright browser session.

Plain-English Explanation JavaScript never checks what *type* of value a variable holds — a variable can be a string one moment and a number the next, and JavaScript won't complain until something actually breaks while running. TypeScript adds a checking step (compilation) where it reads your code and verifies that types are used consistently, flagging problems in your editor immediately, before you ever run `npx playwright test` or `npx cypress run`.

Playwright Example

```
// This is caught instantly in your editor — 'members' isn't a number
function formatCohortSize(size: number): string {
  return `${size} members`;
}
formatCohortSize('two hundred'); // red squiggly line: Argument of type
  ↳ 'string' is not assignable to parameter of type 'number'
```

Cypress Example

```
// Same idea inside a custom command
Cypress.Commands.add('waitForCohortSize', (expectedSize: number) => {
  cy.get('[data-testid="cohort-count"]').should('contain.text',
    ↳ `${expectedSize}`);
});
cy.waitForCohortSize('250'); // caught before you even run the test
```

Common Mistake Assuming TypeScript changes how your code *runs* — it doesn't. TypeScript is stripped away entirely before the code executes; it only checks your code, it never adds runtime behavior. A type error is always a compile-time warning, never something you'll see in a browser console during a test run.

Quick Practice Tip Deliberately write one line of “wrong” code (like passing a string into a function expecting a number) in any test file and watch what your editor does — that red squiggly line is TypeScript working exactly as intended.

1.2 Type Inference vs. Explicit Annotation

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work Most of the time you won't write type annotations at all — TypeScript figures out (infers) the type from the value itself. Knowing when TypeScript can infer vs. when you must be explicit saves you from over-annotating every single line, which is a common beginner habit that makes code harder to read, not safer.

Plain-English Explanation If you write `const cohortName = 'Diabetes Cohort'`, TypeScript looks at the value `'Diabetes Cohort'` and infers the type is `string` — you never had to write

: string yourself. You only need explicit annotations when TypeScript has nothing to infer from yet, like an empty array, or a function parameter (function parameters are never inferred from usage — you must annotate them).

Playwright Example

```
// Inferred – TypeScript knows these are string and number, no annotation
↪ needed
const dashboardUrl = '/dashboards/cohort-explorer'; // inferred as string
const retryCount = 3; // inferred as number

// Explicit – required, because a function parameter has no value to infer from
async function navigateToCohort(page: Page, cohortId: string): Promise<void> {
  await page.goto(`/dashboards/cohort/${cohortId}`);
}
```

Cypress Example

```
// Inferred
const roleName = 'analyst'; // inferred as string

// Explicit – needed for the parameter, since Cypress can't guess what you'll
↪ pass
Cypress.Commands.add('loginAs', (role: string) => {
  cy.request('POST', '/api/auth/login', { username: role });
});
```

Common Mistake Writing `const cohortName: string = 'Diabetes Cohort'` everywhere out of caution — this isn't wrong, but it's unnecessary noise. Beginners often over-annotate variables that TypeScript would infer perfectly well on its own; save explicit annotations for function parameters, return types on complex functions, and cases where inference genuinely can't help (like an empty array you'll fill later).

Quick Practice Tip Hover your mouse over any variable in VS Code without an annotation — the tooltip that pops up showing its inferred type is TypeScript telling you exactly what it figured out.

1.3 The Basic Types You'll See Every Day: string, number, boolean, arrays

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work These four cover the overwhelming majority of values in test code — URLs and names (string), counts and timeouts (number), flags like “is this checkbox checked” (boolean), and lists of test data (arrays).

Plain-English Explanation string, number, and boolean are exactly what they sound like. Arrays are written as `type[]` — so a list of strings is `string[]`, a list of numbers is `number[]`. This is the same array you already know from JavaScript; TypeScript just lets you declare what kind of things live inside it.

Playwright Example

```
const cohortName: string = 'Active Diabetes Patients';
const memberCount: number = 250;
const isArchived: boolean = false;
```

```
const roleNames: string[] = ['admin', 'analyst', 'auditor'];

for (const role of roleNames) {
  await page.goto(`/login?role=${role}`);
}
```

Cypress Example

```
const testCohortIds: string[] = [];

before(() => {
  const cohortNames: string[] = ['Cohort A', 'Cohort B', 'Cohort C'];
  cohortNames.forEach((name) => {
    cy.createTestCohort({ name }).then((id: string) => testCohortIds.push(id));
  });
});
```

Common Mistake Writing `Array<string>` when `string[]` would do — both mean exactly the same thing, but `string[]` is the more common, more readable style you’ll see in almost all Playwright and Cypress codebases; use `Array<T>` mainly when the type itself is more complex and the square-bracket version would look confusing.

Quick Practice Tip Whenever you see `Type[]` in someone else’s code, mentally read it as “a list of Type” — that reading works for every array type you’ll ever encounter.

1.4 any, unknown, and never — the Three Types That Confuse Everyone Coming From JavaScript

Level: Core | **You’ll Use This:** Occasionally, but getting it wrong causes real bugs

Why It Matters For Your Work `any` is the type that quietly turns off TypeScript’s checking for a value — reaching for it too often defeats the entire point of using TypeScript, and it’s the single most common beginner habit that experienced reviewers will flag in a PR.

Plain-English Explanation `any` means “TypeScript, stop checking this — trust me.” It’s an escape hatch, and every time you use it, you’ve created a blind spot where a real bug could hide undetected. `unknown` is the safer alternative — it also means “I don’t know the type yet,” but TypeScript forces you to *check* what it actually is before you’re allowed to use it, unlike `any` which lets you do anything with no checks at all. `never` is rarer — it means a value that can genuinely never occur (like a function that always throws an error instead of returning).

Playwright Example

```
// any – dangerous, TypeScript won't catch mistakes here at all
async function handleApiResponse(data: any) {
  return data.memberCount.toUpperCase(); // no error shown, but this will
  ↪ crash at runtime – numbers don't have toUpperCase()
}

// unknown – safer, forces you to check before using it
async function handleApiResponseSafely(data: unknown) {
  if (typeof data === 'object' && data !== null && 'memberCount' in data) {
    console.log((data as { memberCount: number }).memberCount);
  }
}
```

```
}  
}
```

Cypress Example

```
// A cy.request() response body is typed as 'any' by default – this is a real,  
↪ common gotcha  
cy.request('GET', '/api/cohorts/123').then((response) => {  
  const cohort = response.body; // typed as 'any' – no safety here unless you  
  ↪ annotate it  
  console.log(cohort.memberCount); // works, but TypeScript can't warn you if  
  ↪ this field doesn't exist  
});  
  
// Better – tell TypeScript the actual shape you expect  
interface CohortResponse { id: string; memberCount: number; }  
cy.request('GET', '/api/cohorts/123').then((response) => {  
  const cohort = response.body as CohortResponse;  
  console.log(cohort.memberCount); // now TypeScript actually knows this field  
  ↪ exists  
});
```

Common Mistake Reaching for any the moment TypeScript complains about something, just to make the red squiggly line go away — this is the single habit to unlearn first, because it silently disables the safety net you’re trying to build.

Quick Practice Tip Every time you’re about to type : any, pause and ask “do I actually know the real shape of this data?” — in test automation, you almost always do (it’s your own API response, your own test data), so define it properly instead.

SECTION 2 — Objects, Interfaces & Type Aliases

2.1 Interfaces — Describing the Shape of Your Test Data

Level: Foundational | **You’ll Use This:** Constantly

Why It Matters For Your Work Almost everything you deal with in test automation is an object with a specific shape — a cohort, a user, an API response, a test config. An interface names that shape once so every function, fixture, and test that touches it agrees on what fields exist.

Plain-English Explanation An interface is a named description of what properties an object must have, and what type each property is. Once defined, you can use that interface name anywhere you’d otherwise have to describe the same object shape all over again.

Playwright Example

```
interface Cohort {  
  id: string;  
  name: string;  
  memberCount: number;  
  status: string;  
}
```

```

async function assertCohortLoaded(page: Page, cohort: Cohort) {
  await expect(page.getByText(cohort.name)).toBeVisible();
  await expect(page.getByTestId('cohort-count')).toHaveText(`${cohort.
    ↪ memberCount}
    ↪ members`);
}

```

Cypress Example

```

interface Cohort {
  id: string;
  name: string;
  memberCount: number;
  status: string;
}

Cypress.Commands.add('createTestCohort', (params: Partial<Cohort>) => {
  return cy
    .request('POST', '/api/cohorts', params)
    .its('body') as Cypress.Chainable<Cohort>;
});

```

Common Mistake Repeating the same object shape inline in five different function signatures instead of naming it once with an interface — the moment the shape changes (a new field added), you have to hunt down and fix five separate places instead of one.

Quick Practice Tip The moment you type out the same { id: string; name: string; ... } shape a second time anywhere in your codebase, that's your signal to extract it into a named interface.

2.2 Optional Properties (?) and readonly

Level: Core | **You'll Use This:** Frequently

Why It Matters For Your Work Real test data is often incomplete — you don't always know a cohort's archivedAt date until it's actually archived. The ? marks a property as optional, which matches reality far better than pretending every field always exists.

Plain-English Explanation Adding ? after a property name in an interface means “this property might be present, or might be undefined — both are fine.” readonly means the opposite kind of safety — it says “once this is set, nothing in the code is allowed to change it later,” which is useful for values like an ID that should never be reassigned after creation.

Playwright Example

```

interface Cohort {
  readonly id: string;           // set once at creation, never reassigned
  name: string;
  memberCount: number;
  archivedAt?: string;          // optional — most cohorts are never archived
}

function describeCohort(cohort: Cohort): string {
  return cohort.archivedAt
}

```

```

    ? `${cohort.name} (archived on ${cohort.archivedAt})`
    : `${cohort.name} (active)`;
}

```

Cypress Example

```

interface TestUser {
  readonly username: string;
  password: string;
  role?: 'admin' | 'analyst'; // optional – defaults to a standard role if not
  ↪ given
}

Cypress.Commands.add('loginAs', (user: TestUser) => {
  const role = user.role ?? 'analyst'; // fall back if role wasn't provided
  cy.request('POST', '/api/auth/login', { username: user.username, password:
  ↪ user.password, role });
});

```

Common Mistake Marking a field optional (?) just to silence an error, when it should actually always be present — this hides a real bug (data that’s genuinely missing when it shouldn’t be) behind a type that says “this is fine to be missing.”

Quick Practice Tip Before adding ? to a field, ask: “is it *genuinely* sometimes absent, or am I just working around incomplete test data setup?” If it’s the latter, fix the setup instead of loosening the type.

2.3 Type Aliases — and When They’re Better Than an Interface

Level: Core | **You’ll Use This:** Frequently

Why It Matters For Your Work type and interface look similar and often do the same job, but type is the only option for a few things you’ll use constantly in test automation — like naming a specific set of allowed string values (a role, a status).

Plain-English Explanation An interface describes the shape of an *object*. A type alias can name literally any type — an object shape (like an interface can), but also a union of specific allowed values, a function signature, or a combination of other types. For plain object shapes, either works and it’s mostly a style choice; for naming a fixed set of allowed strings, you need type.

Playwright Example

```

// type alias for a fixed set of allowed values – interface can't do this
type Role = 'admin' | 'analyst' | 'auditor';

// type alias for an object shape – works exactly like an interface would here
type Cohort = {
  id: string;
  name: string;
};

async function loginAs(page: Page, role: Role) {

```

```
await page.goto(`/login?role=${role}`);
}
```

Cypress Example

```
type Role = 'admin' | 'analyst' | 'auditor';

Cypress.Commands.add('loginAs', (role: Role) => {
  // TypeScript will now reject any string that isn't exactly 'admin',
  //   ↳ 'analyst', or 'auditor'
  cy.request('POST', '/api/auth/login', { role });
});

cy.loginAs('superadmin'); // caught immediately — 'superadmin' isn't one of
  ↳ the allowed values
```

Common Mistake Using type `Role = string` instead of the specific union `'admin' | 'analyst' | 'auditor'` — this technically works but throws away the exact protection you wanted, since now any string at all is accepted, including typos.

Quick Practice Tip Anywhere you have a fixed, known list of allowed string values (roles, statuses, environments), write them as a union type — it turns typos into instant compile errors instead of confusing runtime bugs.

SECTION 3 — Functions in TypeScript

3.1 Typing Function Parameters and Return Values

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work Every helper function, every custom command, every Page Object method needs its inputs and outputs typed — this is where TypeScript catches the most real bugs in day-to-day test writing, like passing a cohort object where a cohort ID string was expected.

Plain-English Explanation You annotate each parameter with `: Type` and, optionally, the return value after the parameter list with `: ReturnType`. TypeScript will then reject any call that passes the wrong type of argument, and flag your function's body if it tries to return something that doesn't match the declared return type.

Playwright Example

```
function buildDashboardUrl(cohortId: string, tab: string): string {
  return `/dashboards/cohort/${cohortId}?tab=${tab}`;
}

async function navigateToTab(page: Page, cohortId: string, tab: string):
  ↳ Promise<void> {
  await page.goto(buildDashboardUrl(cohortId, tab));
}

navigateToTab(page, 123, 'summary'); // caught — 123 is a number, not a string
```

Cypress Example

```
function buildDashboardUrl(cohortId: string, tab: string): string {
  return `/dashboards/cohort/${cohortId}?tab=${tab}`;
}

Cypress.Commands.add('visitCohortTab', (cohortId: string, tab: string) => {
  cy.visit(buildDashboardUrl(cohortId, tab));
});
```

Common Mistake Leaving off the return type on functions that are more than a couple of lines long — TypeScript will infer it, but writing it explicitly on anything non-trivial makes it much easier for a teammate (or you, in six months) to understand what a function promises to give back without reading its whole body.

Quick Practice Tip As a rule of thumb: always annotate parameters (TypeScript can never infer these), and annotate the return type explicitly once a function’s body is more than a one-liner.

3.2 Optional and Default Parameters

Level: Core | **You’ll Use This:** Frequently

Why It Matters For Your Work Test helper functions very often need a “usually this, but sometimes override it” pattern — like a cohort-seeding helper that defaults to size 100 unless you specifically ask for a different size.

Plain-English Explanation An optional parameter (marked with `?`) may be omitted entirely when calling the function, and will be undefined inside the function if not provided. A default parameter gives it an actual fallback value instead of undefined, using `= value` in the parameter list — this is usually the better choice, since your code doesn’t have to separately check “did they pass this or not.”

Playwright Example

```
async function seedCohort(apiClient: CohortApiClient, size: number = 100,
  ↪ name?: string): Promise<string> {
  return apiClient.createTestCohort({ size, name: name ?? `test-${Date.now()}`
  ↪ });
}

await seedCohort(apiClient); // uses size = 100, generated name
await seedCohort(apiClient, 250); // uses size = 250, generated name
await seedCohort(apiClient, 250, 'VIP'); // both provided explicitly
```

Cypress Example

```
Cypress.Commands.add('createTestCohort', (size: number = 100, name?: string)
  ↪ => {
  return cy.request('POST', '/api/cohorts', { size, name: name ??
  ↪ `test-${Date.now()}` }).its('body.id');
});

cy.createTestCohort(); // size defaults to 100
cy.createTestCohort(250); // explicit size
```

Common Mistake Putting a required parameter *after* an optional or default one — TypeScript won't allow this, because there'd be no way to skip the optional one while still providing the required one after it. Required parameters must always come first.

Quick Practice Tip When in doubt between optional (?) and default (= value), prefer default parameters — they eliminate an entire class of “forgot to handle the undefined case” bugs inside the function body.

3.3 Typing Callback Functions (the Pattern Behind `page.route()` and `cy.intercept()`)

Level: Core | **You'll Use This:** Frequently, often without realizing it

Why It Matters For Your Work Both Playwright's `page.route()` and many Cypress patterns pass you a callback function as an argument — understanding how that callback itself gets typed is what makes autocomplete work inside it, and what stops you from accidentally using the wrong method on the object it hands you.

Plain-English Explanation A callback is just a function you pass as an argument to another function, which then calls it for you later. TypeScript can type *what a callback receives* and *what it must return*, and libraries like Playwright ship those types already built in — you don't write them yourself, you just get the benefit of autocomplete and error-checking because Playwright's own code declares them.

Playwright Example

```
// Playwright already knows page.route()'s callback receives a Route object –
// that's why 'route.' triggers autocomplete showing fulfill(), abort(),
// ↪ continue()
await page.route('**/api/cohorts', (route) => {
  route.fulfill({ status: 200, body: JSON.stringify([]) });
});

// You can still add your own explicit annotation if you want to be clear
// ↪ about it
import { Route } from '@playwright/test';
await page.route('**/api/cohorts', (route: Route) => {
  route.fulfill({ status: 200, body: JSON.stringify([]) });
});
```

Cypress Example

```
// cy.intercept()'s callback receives a request object with typed properties
cy.intercept('POST', '/api/cohorts', (req) => {
  req.reply({ statusCode: 201, body: { id: 'test-123' } });
});
```

Common Mistake Writing `(route: any) => { ... }` because you're not sure what type `route` should be — almost every Playwright/Cypress callback parameter already has a correct type provided by the library itself; you rarely need to write it yourself, and doing so with `any` throws away autocomplete and safety for no reason.

Quick Practice Tip If you're ever tempted to type a callback parameter as `any`, first just remove the annotation entirely and let TypeScript infer it from context — for library callbacks like these, it almost always infers correctly on its own.

SECTION 4 — Async/Await and Promises

4.1 Why Almost Everything in Playwright Is async

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work This is the single most important concept for Playwright specifically — get it wrong and your tests will report false passes, click nothing, and generally behave in confusing ways that have nothing to do with your actual test logic.

Plain-English Explanation A Promise represents a value that isn't available yet, but will be at some point (like a browser action that takes time to complete). `async` marks a function as one that will use `await` inside it and automatically returns a Promise itself. `await` pauses execution *within that function* until the Promise resolves, then continues with the actual resolved value — without it, your code races ahead to the next line before the browser action has actually finished.

Playwright Example

```
// Every test function is async, and every Playwright action needs await in
↪ front of it
test('applies filter and shows results', async ({ page }) => {
  await page.goto('/dashboards/cohort-explorer');
  await page.getByLabel('Age range').fill('40-60');
  await page.getByRole('button', { name: 'Apply' }).click();
  await expect(page.getByTestId('cohort-count')).toBeVisible();
});
```

Cypress Example

```
// Cypress deliberately does NOT use async/await for its own commands — this
↪ is different from Playwright!
it('applies filter and shows results', () => {
  cy.visit('/dashboards/cohort-explorer'); // no await — Cypress queues this
  ↪ internally instead
  cy.get('[data-testid="age-range-input"]').type('40-60');
  cy.get('[data-testid="apply-btn"]').click();
  cy.get('[data-testid="cohort-count"]').should('be.visible');
});
```

Common Mistake Forgetting `await` in front of a Playwright action — the test doesn't crash, it just moves on to the next line immediately, often causing the *next* action to fail confusingly (clicking something that hasn't loaded yet), which looks like an unrelated bug rather than a missing `await`.

Quick Practice Tip In a Playwright test file, if a line calls a `page.` or `expect().to...` method and doesn't start with `await`, that's almost always a bug — most editors/linters can even flag this automatically (the `eslint-plugin-playwright` package specifically checks for missing awaits).

4.2 The Cypress Exception — Why cy. Commands Don't Use await

Level: Core | **You'll Use This:** Constantly, if you write Cypress

Why It Matters For Your Work This is the single most common source of confusion for someone who knows Playwright's async pattern and then writes Cypress — trying to `await cy.get(...)` doesn't do what you'd expect, and understanding why prevents a whole category of confusing bugs.

Plain-English Explanation Cypress commands are deliberately not real Promises — they use an internal command queue (covered in the Cypress handbook's architecture section) instead. This means you never write `await` before a `cy.` command, and you never write your test function as `async`. To get an actual resolved value out of a Cypress command, you use `.then()` instead, which is Cypress's own mechanism for “give me the value once this command finishes.”

Playwright Example (for contrast)

```
// Playwright — real Promises, real await
const memberCount = await page.getByTestId('cohort-count').textContent();
console.log(memberCount); // the actual string value, available right here
```

Cypress Example

```
// Cypress — NOT a real Promise, no await, use .then() to access the resolved
// value
cy.get('[data-testid="cohort-count"]').invoke('text').then((memberCount) => {
  console.log(memberCount); // the actual string value, available inside this
  // callback only
});

// WRONG — this does not work the way it looks like it should
const memberCount = await
  cy.get('[data-testid="cohort-count"]').invoke('text'); // do not do this
```

Common Mistake Writing `async/await` around Cypress commands because it “looks similar” to Playwright — Cypress will not throw an obvious error for this in every case, but the value you get won't be what you expect, since `cy.get()` doesn't return a native Promise that `await` knows how to properly unwrap.

Quick Practice Tip If you're switching between Playwright and Cypress test files in the same day, use this mental rule: “Playwright → `await` everything. Cypress → `await` nothing, use `.then()` when you need the actual value.”

4.3 Typing a Function That Returns a Promise

Level: Core | **You'll Use This:** Frequently, especially in helper functions and API clients

Why It Matters For Your Work Your own helper functions (API clients, seeding utilities) will often be `async`, and knowing how to correctly write their return type helps both TypeScript and your teammates understand exactly what comes back and when it's actually ready to use.

Plain-English Explanation An `async` function's declared return type must be wrapped in `Promise<...>` — if the function returns a `string` once its internal `await`s finish, its type signature is `Promise<string>`, not just `string`. This is because calling an `async` function always

gives you back a Promise immediately; the actual string only becomes available once you await that Promise.

Playwright Example

```
class CohortApiClient {
  constructor(private request: APIRequestContext) {}

  async createTestCohort(params: { size: number }): Promise<string> {
    const response = await this.request.post('/api/cohorts', { data: params });
    const body = await response.json();
    return body.id; // this is a string – but the function's declared type is
    ↪ Promise<string>
  }
}

const cohortId: string = await apiClient.createTestCohort({ size: 100 }); //
↪ await unwraps the Promise
```

Cypress Example

```
// Cypress commands don't use Promise<T> the same way – instead they use
↪ Cypress.Chainable<T>
Cypress.Commands.add('createTestCohort', (params: { size: number }):
↪ Cypress.Chainable<string> => {
  return cy.request('POST', '/api/cohorts', params).its('body.id');
});
```

Common Mistake Writing function `createTestCohort(): string` (forgetting the `Promise<>` wrapper) on an async function — TypeScript will actually give you an error here, since an async function can never truly return a bare string, only a `Promise<string>`, so this mistake usually gets caught immediately rather than silently causing a bug.

Quick Practice Tip Whenever you write the word `async` in front of a function, immediately ask “what does this eventually resolve to?” and wrap that answer in `Promise<...>` for the return type.

SECTION 5 — Classes for Page Objects

5.1 Class Basics — Constructor, Properties, and Methods

Level: Core | **You'll Use This:** Constantly, if you write Page Objects

Why It Matters For Your Work Page Objects — the reusable classes you've likely already been writing for dashboards, forms, and modals — are exactly what a TypeScript class is for. Understanding the syntax properly (instead of copy-pasting a template) means you can actually design them well, not just make them compile.

Plain-English Explanation A class is a blueprint for creating objects that bundle related data (properties) and behavior (methods) together. The constructor is a special method that runs once when you create (new) an instance of the class — it's where you typically receive and store whatever the object needs to do its job, like the Playwright page object.

Playwright Example

```

class CohortExplorerPage {
  constructor(private page: Page) {} // stores 'page' as a property
    ↪ automatically

  async goto(cohortId: string): Promise<void> {
    await this.page.goto(`/dashboards/cohort/${cohortId}`);
  }

  get memberCount(): Locator {
    return this.page.getByTestId('cohort-count');
  }
}

const cohortExplorer = new CohortExplorerPage(page); // 'new' creates an
    ↪ instance
await cohortExplorer.goto('123');
await expect(cohortExplorer.memberCount).toBeVisible();

```

Cypress Example

```

class CohortExplorerPage {
  visit(cohortId: string): this {
    cy.visit(`/dashboards/cohort/${cohortId}`);
    return this; // returning 'this' lets you chain method calls
  }

  get memberCount() {
    return cy.get('[data-testid="cohort-count"]');
  }
}

const cohortExplorer = new CohortExplorerPage();
cohortExplorer.visit('123');
cohortExplorer.memberCount.should('be.visible');

```

Common Mistake Forgetting the private keyword in the constructor’s page: Page parameter — writing just constructor(page: Page) stores nothing automatically; you’d have to manually write this.page = page yourself. The shortcut constructor(private page: Page) does both — declares the property *and* assigns it — in one line, which is why you’ll see it everywhere in Playwright Page Objects.

Quick Practice Tip Read constructor(private page: Page) as “automatically create a private property called page, typed as Page, and set it from whatever gets passed in when this class is instantiated” — that one line is doing more than it looks like.

5.2 Access Modifiers — private, public, and readonly

Level: Core | **You’ll Use This:** Frequently

Why It Matters For Your Work Deciding what a Page Object exposes to the outside (public) versus keeps to itself (private) is what makes a well-designed Page Object easy to use correctly and hard to misuse — exactly the “expose state, don’t expose implementation detail” principle from good Page Object design.

Plain-English Explanation `public` (the default if you write nothing) means any code outside the class can access that property or method. `private` means only code *inside* the class itself can access it — an attempt to reach it from outside is a compile error, not just bad practice. `readonly` means a property can be set once (usually in the constructor) and never reassigned afterward.

Playwright Example

```
class CohortExplorerPage {
  private readonly page: Page; // only this class can touch 'page' directly,
    ↪ and it's set once

  constructor(page: Page) {
    this.page = page;
  }

  // public by default — this is the class's actual usable interface
  async goto(cohortId: string): Promise<void> {
    await this.page.goto(`/dashboards/cohort/${cohortId}`);
  }
}

const cohortExplorer = new CohortExplorerPage(page);
cohortExplorer.page; // ERROR — 'page' is private, can't be accessed from
    ↪ outside the class
```

Cypress Example

```
class CohortApiClient {
  private readonly baseUrl: string;

  constructor(baseUrl: string) {
    this.baseUrl = baseUrl;
  }

  createTestCohort(params: { size: number }) {
    return cy.request('POST', `${this.baseUrl}/api/cohorts`,
      ↪ params).its('body.id');
  }
}
```

Common Mistake Making everything `public` (or leaving off the modifier, which defaults to `public`) out of not wanting to deal with access errors — this defeats the purpose of a class as a clean interface, since now every internal detail is exposed and can be reached, modified, or misused from anywhere.

Quick Practice Tip Default new properties to `private` first; only widen to `public` once you've confirmed something genuinely needs to be used from outside the class — this is easier than the reverse (finding and fixing all the places that came to depend on something that should have stayed hidden).

5.3 Composing Page Objects — a Class That Contains Other Classes

Level: Core | **You'll Use This:** Frequently in real dashboards

Why It Matters For Your Work A real dashboard isn't one flat Page Object — it's made of a filter panel, a data table, an export modal. Composing smaller classes inside a bigger one is how you avoid the "2,000-line God Page Object" problem covered in both automation handbooks.

Plain-English Explanation A class can have properties that are themselves *instances of other classes*, created inside its own constructor. This lets a `DashboardPage` class delegate specific regions of the UI to their own focused classes, while still giving you one convenient entry point.

Playwright Example

```
class FilterPanel {
  constructor(private page: Page) {}
  async setAgeRange(range: string): Promise<void> {
    await this.page.getByLabel('Age range').fill(range);
  }
}

class CohortTable {
  constructor(private page: Page) {}
  get rowCount(): Promise<number> {
    return this.page.getByTestId('cohort-row').count();
  }
}

class DashboardPage {
  readonly filterPanel: FilterPanel;
  readonly cohortTable: CohortTable;

  constructor(page: Page) {
    this.filterPanel = new FilterPanel(page);
    this.cohortTable = new CohortTable(page);
  }
}

const dashboard = new DashboardPage(page);
await dashboard.filterPanel.setAgeRange('40-60');
```

Cypress Example

```
class FilterPanel {
  setAgeRange(range: string): this {
    cy.get('[data-testid="age-range-input"]').clear().type(range);
    return this;
  }
}

class DashboardPage {
  readonly filterPanel = new FilterPanel(); // can also assign directly, no
  ⇨ constructor needed here

  visit(): this {
    cy.visit('/dashboards/cohort-explorer');
    return this;
  }
}
```

```
const dashboard = new DashboardPage();
dashboard.visit().filterPanel.setAgeRange('40-60');
```

Common Mistake Putting every method for every UI region directly on one big class instead of composing smaller focused classes — this is exactly the merge-conflict and unreadability problem covered in the enterprise-framework sections of both automation handbooks.

Quick Practice Tip If a Page Object class is growing past roughly 150-200 lines, that’s usually the signal to split off a chunk of its methods into their own focused class and compose it in, the same way FilterPanel and CohortTable were split out above.

SECTION 6 — Union Types, Narrowing & Modern Syntax Shortcuts

6.1 Union Types — “This or That, Nothing Else”

Level: Core | **You’ll Use This:** Constantly

Why It Matters For Your Work You already met this briefly in Section 2 with Role. Union types are how you tell TypeScript “this value can only ever be one of these specific things” — roles, statuses, environments, browser names — which turns typos into instant errors instead of confusing runtime bugs three steps into a test.

Plain-English Explanation A union type is written with a pipe (|) between each allowed option. The value must match exactly one of them — nothing else is accepted.

Playwright Example

```
type Environment = 'dev' | 'staging' | 'prod-smoke';

function getBaseUrl(env: Environment): string {
  const urls: Record<Environment, string> = {
    dev: 'http://localhost:3000',
    staging: 'https://staging.example.com',
    'prod-smoke': 'https://app.example.com',
  };
  return urls[env];
}

getBaseUrl('production'); // caught immediately — 'production' isn't one of
↪ the three allowed values
```

Cypress Example

```
type CohortStatus = 'active' | 'archived' | 'pending';

Cypress.Commands.add('createTestCohort', (status: CohortStatus, name: string)
  ↪ => {
  cy.request('POST', '/api/cohorts', { status, name });
});
```

```
cy.createTestCohort('archived', 'Test Cohort'); // fine
cy.createTestCohort('deleted', 'Test Cohort'); // caught - 'deleted' isn't
↳ allowed
```

Common Mistake Widening a union type back to plain string the moment it becomes inconvenient (like when passing it through a generic helper function) — this quietly throws away the exact protection the union was giving you; it’s usually better to fix the helper function’s typing than to loosen the union.

Quick Practice Tip Any time you find yourself writing a comment like `// must be 'admin', 'analyst', or 'auditor'` above a string parameter, that comment should become an actual union type instead — comments don’t get enforced, union types do.

6.2 Narrowing — Checking Which Option a Union Actually Is

Level: Core | **You’ll Use This:** Frequently

Why It Matters For Your Work Once you have a union type or an unknown value, you often need to handle each possibility differently — narrowing is how TypeScript lets you safely check “which one is this, right now” before acting on it.

Plain-English Explanation Narrowing means using a check (like `typeof`, `in`, or a simple `if/else`) that TypeScript itself understands well enough to know, inside that branch of code, exactly which type you’re dealing with — giving you full autocomplete and safety for that specific type from that point onward.

Playwright Example

```
type JobStatus = 'pending' | 'complete' | 'failed';

async function handleExportStatus(status: JobStatus, page: Page) {
  if (status === 'complete') {
    await expect(page.getByText('Export ready')).toBeVisible();
  } else if (status === 'failed') {
    await expect(page.getByText('Export failed')).toBeVisible();
  } else {
    await expect(page.getByTestId('loading-spinner')).toBeVisible();
  }
}
```

Cypress Example

```
interface SuccessResponse { status: 'success'; cohortId: string; }
interface ErrorResponse { status: 'error'; message: string; }
type ApiResponse = SuccessResponse | ErrorResponse;

cy.request('POST', '/api/cohorts').then((response) => {
  const body = response.body as ApiResponse;
  if (body.status === 'success') {
    cy.visit(`/dashboards/cohort/${body.cohortId}`); // TypeScript knows
    ↳ cohortId exists here
  } else {
    cy.log(`Failed: ${body.message}`); // TypeScript knows message exists here
    ↳ instead
```

```
}  
});
```

Common Mistake Trying to access a property that only exists on *one* branch of a union before actually narrowing it — TypeScript correctly refuses this, because it can't guarantee the property exists until you've checked which specific type you're dealing with.

Quick Practice Tip If TypeScript says a property “does not exist on type X | Y,” that's almost always a sign you need an `if` check narrowing which one it actually is, before trying to use a property that only belongs to one of them.

6.3 Optional Chaining (?.) and Nullish Coalescing (??)

Level: Core | **You'll Use This:** Constantly

Why It Matters For Your Work Test data is often incomplete or nested deeply (a cohort that might not have a `lastExport` object yet, which might not have a `completedAt` field). These two operators are the modern, safe shortcut for handling that without writing a wall of `if` checks.

Plain-English Explanation `?.` safely accesses a property that might not exist — if anything along the chain is `null` or `undefined`, the whole expression short-circuits to `undefined` instead of throwing an error. `??` gives a fallback value specifically when the left side is `null` or `undefined` (unlike `||`, which also falls back on falsy-but-valid values like `0` or `''`, which is usually not what you want).

Playwright Example

```
interface Cohort {  
  name: string;  
  lastExport?: { completedAt?: string };  
}  
  
function getLastExportTime(cohort: Cohort): string {  
  return cohort.lastExport?.completedAt ?? 'Never exported';  
}  
  
// Without optional chaining, this would need a much longer nested check:  
// if (cohort.lastExport && cohort.lastExport.completedAt) { ... } else {  
  ↪ 'Never exported' }
```

Cypress Example

```
cy.request('GET', '/api/cohorts/123').then((response) => {  
  const memberCount = response.body?.stats?.memberCount ?? 0;  
  cy.log(`Member count: ${memberCount}`);  
});
```

Common Mistake Using `||` instead of `??` for a fallback when zero or an empty string are valid real values — `cohort.memberCount || 100` would incorrectly fall back to `100` even when `memberCount` is genuinely and correctly `0`, since `0` is falsy; `cohort.memberCount ?? 100` only falls back when the value is actually `null/undefined`.

Quick Practice Tip Default to `??` over `||` for fallback values in test code — it's very rarely wrong, whereas `||` has a specific, easy-to-miss gotcha with `0`, `''`, and `false`.

SECTION 7 — Generics (Made Simple, Tied Directly to Fixtures)

7.1 What Problem Generics Actually Solve — Before Any Syntax

Level: Core, but explained from zero | **You'll Use This:** Every time you write a Playwright fixture

Why It Matters For Your Work You've almost certainly copy-pasted `test.extend<MyFixtures>({...})` without fully knowing what the `<MyFixtures>` part does. This is the moment that gets fully explained — once it clicks, fixtures stop feeling like magic syntax and start feeling like something you actually understand.

Plain-English Explanation Imagine a function `identity` that just returns whatever you give it back. Without generics, you'd have to write a separate version for every type: `identityString(x: string): string`, `identityNumber(x: number): number`, and so on — endless duplication. A generic lets you write the function *once*, using a placeholder (conventionally named `T`) for “whatever type gets used this time,” and TypeScript fills in the real type automatically based on what you actually pass in.

Playwright Example

```
// Without generics – you'd need one of these per type, which doesn't scale
function identityString(x: string): string { return x; }
function identityNumber(x: number): number { return x; }

// With generics – one function, works for any type, and TypeScript tracks
//   ↪ which one
function identity<T>(x: T): T {
  return x;
}

const a = identity('hello'); // TypeScript infers T = string, so 'a' is a
//   ↪ string
const b = identity(42);      // TypeScript infers T = number, so 'b' is a
//   ↪ number
```

Cypress Example

```
// A generic helper that works for any test-data type, not just cohorts
function pickRandom<T>(items: T[]): T {
  return items[Math.floor(Math.random() * items.length)];
}

const roles: string[] = ['admin', 'analyst', 'auditor'];
const randomRole: string = pickRandom(roles); // T = string here

const cohortIds: number[] = [1, 2, 3];
const randomId: number = pickRandom(cohortIds); // T = number here – same
//   ↪ function, different type
```

Common Mistake Thinking `T` has to specifically mean “type” in some special magic way — it's just a placeholder name, exactly like a regular function parameter name, except it stands

in for a *type* instead of a *value*. You could name it anything (TItem, TValue), though T is the near-universal convention.

Quick Practice Tip Read <T> as “fill in the blank with whatever specific type is actually used this time” — that’s the entire concept; everything else is just applying that idea in different places.

7.2 Generics in Playwright’s `test.extend<Fixtures>()` — Finally Explained

Level: Core | **You’ll Use This:** Every fixture file you write

Why It Matters For Your Work This is the exact pattern from both automation handbooks that you’ve been using. Now that generics themselves make sense, this line stops being a memorized incantation.

Plain-English Explanation `test.extend<MyFixtures>({...})` is a generic function call — `MyFixtures` is the specific type being “filled in” for the placeholder that `test.extend` expects internally. It’s an interface (or type) *you* define, listing every custom fixture’s name and its type, so that everywhere you later write `async ({ myFixture }) => {...}`, TypeScript already knows exactly what `myFixture` is and gives you correct autocomplete for it.

Playwright Example

```
import { test as base } from '@playwright/test';
import { CohortApiClient } from '../api/CohortApiClient';

// This interface IS the 'T' being filled in below – you're naming your own
//   fixtures' shapes
interface MyFixtures {
  apiClient: CohortApiClient;
  seededCohortId: string;
}

// test.extend<MyFixtures>(…) tells TypeScript: "the fixtures I'm adding
//   match this shape"
export const test = base.extend<MyFixtures>({
  apiClient: async ({ request }, use) => {
    await use(new CohortApiClient(request));
  },
  seededCohortId: async ({ apiClient }, use) => {
    const id = await apiClient.createTestCohort({ size: 50 });
    await use(id);
  },
});

// Now in any spec file, TypeScript knows exactly what these fixtures are
import { test, expect } from '../fixtures';

test('cohort loads correctly', async ({ apiClient, seededCohortId }) => {
  // apiClient is known to be CohortApiClient, seededCohortId is known to be
  //   string –
  //   autocomplete works, and passing the wrong type anywhere here would be an
  //   instant error
```

```
});
```

Common Mistake Skipping the `MyFixtures` interface entirely and just writing fixtures without declaring their types — this still runs fine, but you lose autocomplete and type-checking on every fixture across every spec file that uses it, which defeats a large part of why TypeScript-based Playwright projects are worth the setup.

Quick Practice Tip Whenever you add a brand-new custom fixture, the very first edit should be adding its name and type to the `MyFixtures` interface — write the “what is this” before the “how does it work.”

7.3 A Simpler, Everyday Generic You’ll Actually Write Yourself: A Typed API Helper

Level: Core | **You’ll Use This:** Occasionally, in your own utility functions

Why It Matters For Your Work Fixture typing is something you’ll mostly copy the *pattern* of. Writing your own small generic helper — like a function that fetches any entity by ID and gives you back the correctly-typed result — is where you’ll actually author generics yourself.

Plain-English Explanation You can write your own generic functions the same way `identity<T>` worked earlier, applied to something genuinely useful: a function that fetches JSON and lets the *caller* specify what shape they expect back, instead of the function always returning the unsafe `any`.

Playwright Example

```
async function fetchJson<T>(request: APIRequestContext, url: string):  
  ↪ Promise<T> {  
    const response = await request.get(url);  
    return response.json(); // TypeScript trusts you here – this is a common,  
    ↪ reasonable use of a type assertion internally  
  }  
  
interface Cohort { id: string; memberCount: number; }  
  
// The caller specifies T = Cohort right here, and gets a fully-typed result  
↪ back  
const cohort = await fetchJson<Cohort>(request, '/api/cohorts/123');  
console.log(cohort.memberCount); // autocomplete works, typos in field names  
↪ are caught
```

Cypress Example

```
function requestJson<T>(url: string): Cypress.Chainable<T> {  
  return cy.request('GET', url).its('body') as Cypress.Chainable<T>;  
}  
  
interface Cohort { id: string; memberCount: number; }  
  
requestJson<Cohort>('/api/cohorts/123').then((cohort) => {  
  cy.log(`Members: ${cohort.memberCount}`); // fully typed, no 'any' anywhere  
});
```

Common Mistake Writing the generic helper but then never actually specifying `<Cohort>` when calling it — without providing the type argument (and without enough context for TypeScript to infer it), T falls back to unknown, and you lose the benefit the generic was supposed to give you.

Quick Practice Tip If you write a generic function and find yourself never actually writing `<SpecificType>` when calling it, ask whether the function even needs to be generic — sometimes a fixed, specific type is simpler and clearer than a generic nobody ends up using flexibly.

SECTION 8 — Utility Types You’ll Actually Use

8.1 Partial<T> — “All These Fields, But Optional”

Level: Core | **You’ll Use This:** Frequently in test-data builders

Why It Matters For Your Work Test-seeding functions almost always need to accept “some subset of a cohort’s fields, with sensible defaults for the rest” — `Partial<T>` gives you exactly that shape without redefining every field as optional by hand.

Plain-English Explanation `Partial<Cohort>` takes an existing interface `Cohort` and produces a new type where every field is optional, automatically — you don’t have to go back and add `?` to each field yourself, or maintain a second, separately-written “optional version” of the same interface.

Playwright Example

```
interface Cohort {
  id: string;
  name: string;
  size: number;
  status: string;
}

// Accepts any subset of Cohort's fields – caller only provides what they care
// ↳ about overriding
async function createTestCohort(apiClient: CohortApiClient, overrides:
// ↳ Partial<Cohort> = {}): Promise<string> {
  const defaults: Omit<Cohort, 'id'> = { name: `test-${Date.now()}`, size:
    ↳ 100, status: 'active' };
  return apiClient.createTestCohort({ ...defaults, ...overrides });
}

await createTestCohort(apiClient); // all defaults
await createTestCohort(apiClient, { size: 500 }); // only size
// ↳ overridden
```

Cypress Example

```
interface Cohort { id: string; name: string; size: number; status: string; }

Cypress.Commands.add('createTestCohort', (overrides: Partial<Cohort> = {}) =>
// ↳ {
  const defaults = { name: `test-${Date.now()}`, size: 100, status: 'active' };
  // ...
```

```
return cy.request('POST', '/api/cohorts', { ...defaults, ...overrides
  ↪  }).its('body.id');
});
```

Common Mistake Manually writing out a second interface with every field marked ? instead of just wrapping the original with `Partial<>` — this duplicates the field list, and the two interfaces will inevitably drift out of sync the moment one of them gets updated.

Quick Practice Tip Anywhere you write “a version of this interface, but every field optional,” reach for `Partial<T>` instead of manually retyping the whole shape.

8.2 `Pick<T, Keys>` and `Omit<T, Keys>` — Selecting or Excluding Specific Fields

Level: Core | **You’ll Use This:** Occasionally

Why It Matters For Your Work Sometimes a function only needs a couple of fields from a bigger interface (like just the name and size needed to create a cohort, not its server-generated id) — `Pick` and `Omit` let you derive that narrower shape from the original, instead of writing a whole new interface.

Plain-English Explanation `Pick<Cohort, 'name' | 'size'>` produces a new type containing *only* those two fields from `Cohort`. `Omit<Cohort, 'id'>` does the opposite — it takes everything from `Cohort` *except* the listed field(s). Both keep the derived type automatically in sync if the original interface changes.

Playwright Example

```
interface Cohort {
  id: string;           // server-generated, never provided by the client
  name: string;
  size: number;
  status: string;
}

// The API for CREATING a cohort never includes 'id' – Omit expresses that
↪ precisely
type NewCohortParams = Omit<Cohort, 'id'>;

async function createTestCohort(apiClient: CohortApiClient, params:
  ↪ NewCohortParams): Promise<string> {
  return apiClient.createTestCohort(params);
}

createTestCohort(apiClient, { id: '123', name: 'Test', size: 50, status:
  ↪ 'active' }); // ERROR – 'id' isn't allowed here
```

Cypress Example

```
interface Cohort { id: string; name: string; size: number; status: string; }

// Only need name and size for a quick smoke-test helper – Pick narrows to
↪ exactly that
type QuickCohortParams = Pick<Cohort, 'name' | 'size'>;
```

```
Cypress.Commands.add('quickSeedCohort', (params: QuickCohortParams) => {
  cy.request('POST', '/api/cohorts', { ...params, status: 'active' });
});
```

Common Mistake Writing a brand-new, separately-maintained interface (NewCohortParams { name: string; size: number; status: string; }) instead of deriving it from the original with Omit/Pick — when the original Cohort interface later gains a new field, the manually-written version silently doesn't get it, while the derived version automatically does.

Quick Practice Tip Before writing a “smaller version” of an existing interface by hand, ask whether Pick or Omit already expresses exactly the shape you need — it usually does.

8.3 Record<Keys, ValueType> — Typing an Object Used as a Lookup Map

Level: Core | **You'll Use This:** Occasionally, very useful for role/environment maps

Why It Matters For Your Work You've already seen this pattern in Section 6.1's getBaseUrl function — Record is exactly how you'd type an object that maps a fixed set of keys (like every role, or every environment) to a specific kind of value.

Plain-English Explanation Record<'admin' | 'analyst' | 'auditor', string> describes an object that must have exactly those three keys, each mapping to a string value — TypeScript will actually complain if you're missing one of the required keys, which is a nice safety net for exhaustive lookup tables.

Playwright Example

```
type Role = 'admin' | 'analyst' | 'auditor';

// Record forces you to provide a storageState path for every single role –
// ↪ miss one, get an error
const storageStatePaths: Record<Role, string> = {
  admin: '.auth/admin.json',
  analyst: '.auth/analyst.json',
  auditor: '.auth/auditor.json',
  // forgetting 'auditor' here would be a compile error, not a silent runtime
  // ↪ gap
};

function getStorageState(role: Role): string {
  return storageStatePaths[role];
}
```

Cypress Example

```
type Environment = 'dev' | 'staging' | 'prod-smoke';

const retryCounts: Record<Environment, number> = {
  dev: 0,
  staging: 2,
  'prod-smoke': 2,
};
```

Common Mistake Using a plain `{ [key: string]: string }` index signature instead of `Record<Role, string>` when the keys are actually a known, fixed set — the plain index signature accepts *any* string key and doesn't force you to cover every role, silently allowing a missing entry that `Record` would have caught.

Quick Practice Tip Whenever your object's keys are a fixed union type you already defined elsewhere (roles, environments, statuses), use `Record<ThatUnion, ValueType>` instead of a generic string index — it turns “forgot to add an entry” into a compile error instead of a runtime surprise.

SECTION 9 — Modules, Imports/Exports & Declaration Files

9.1 Named Exports vs. Default Exports

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work Every Page Object, fixture file, and helper you write needs to be imported somewhere else — getting the export/import style consistent across your project avoids a surprising amount of beginner confusion about “why won't this import.”

Plain-English Explanation A named export (`export class Foo {}` or `export function bar() {}`) can have as many per file as you like, and must be imported using the exact same name, in curly braces: `import { Foo } from './foo'`. A default export (`export default class Foo {}`) is limited to one per file, and can be imported under *any* name you choose, without curly braces: `import AnythingYouWant from './foo'`. Most modern Playwright/Cypress codebases lean toward named exports specifically because the import name is enforced to match, which is easier to search for across a codebase.

Playwright Example

```
// pages/CohortExplorerPage.ts
export class CohortExplorerPage { /* ... */ } // named export

// tests/cohort-explorer.spec.ts
import { CohortExplorerPage } from '../pages/CohortExplorerPage'; // must
↪ match the exact name
```

Cypress Example

```
// support/commands/cohorts.ts
export function seedDefaultCohort() { /* ... */ } // named export

// cypress/e2e/cohort.cy.ts
import { seedDefaultCohort } from '../../support/commands/cohorts';
```

Common Mistake Mixing default and named exports inconsistently across a codebase — one file exports its Page Object class as default, another exports as named, and new team members constantly guess wrong at the import syntax. Pick one convention (named exports are the more common modern recommendation) and stick to it everywhere.

Quick Practice Tip If your editor's auto-import feature ever offers to import something under a different name than you expected, that's usually a sign the export was a default export — a strong argument for standardizing on named exports throughout your test codebase.

9.2 Declaring Custom Cypress Commands — the .d.ts File, Finally Explained

Level: Core | **You'll Use This:** Every custom command you write

Why It Matters For Your Work You've been shown this pattern throughout the Cypress handbooks. Now it gets explained properly: why it's needed, and exactly what each part is doing.

Plain-English Explanation When you call `Cypress.Commands.add('loginAs', ...)`, you're teaching Cypress *at runtime* that this command exists — but TypeScript does its checking *before* runtime, by reading your code statically, and it has no way to know about a command that only gets registered while the program is running. A `.d.ts` (declaration) file is where you tell TypeScript directly, ahead of time: “trust me, `cy.loginAs()` will exist, and here's exactly what it takes and returns.” This is called “augmenting” (adding onto) Cypress's own built-in Chainable interface.

Cypress Example

```
// cypress/support/index.d.ts
declare namespace Cypress {
  interface Chainable {
    // This line is what makes cy.loginAs('admin') type-check correctly,
    // ↪ everywhere it's called
    loginAs(role: 'admin' | 'analyst' | 'auditor'): Chainable<void>;
  }
}
```

```
// cypress/support/commands/auth.ts — the ACTUAL runtime implementation
Cypress.Commands.add('loginAs', (role: 'admin' | 'analyst' | 'auditor') => {
  cy.request('POST', '/api/auth/login', { role });
});
```

```
// any spec file — now this both runs correctly AND type-checks correctly
cy.loginAs('admin'); // fine
cy.loginAs('unknown'); // caught by TypeScript, thanks to the declaration file
```

Common Mistake Writing the `Cypress.Commands.add(...)` implementation but forgetting the matching `.d.ts` declaration — the command works perfectly at runtime (Cypress doesn't need the declaration to actually function), but every call site shows a TypeScript error, which beginners often “fix” with `// @ts-ignore` instead of writing the five-line declaration that actually solves it.

Quick Practice Tip Treat writing a custom command as a two-file habit, always: the `.ts` file with the actual logic, and one matching entry added to your `index.d.ts`. Neither is complete without the other.

9.3 Importing Playwright's Own Types — Page, Locator, APIRequestContext

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work Every Page Object constructor, every helper function, needs to reference Playwright's own types — knowing where they come from and how to import them removes a small but constant point of friction.

Plain-English Explanation Playwright ships its own TypeScript type definitions inside the `@playwright/test` package itself — you don't write these types yourself, you just import them by name, exactly like importing any other value, except these are types, used only for annotations, and disappear entirely once compiled.

Playwright Example

```
import { Page, Locator, APIRequestContext, expect } from '@playwright/test';

class CohortExplorerPage {
  constructor(private page: Page) {} // Page comes directly from the
    ↪ Playwright package

  get memberCount(): Locator { // so does Locator
    return this.page.getByTestId('cohort-count');
  }
}

class CohortApiClient {
  constructor(private request: APIRequestContext) {} // and APIRequestContext
}
```

Common Mistake Typing a Page Object's constructor parameter as any instead of importing and using Page, because it's not immediately obvious where the Page type comes from — it's always `@playwright/test`, the exact same package your test files already import test and expect from.

Quick Practice Tip Whenever you're not sure what type something in Playwright is, hover over it in a place where it's already correctly used (like the page parameter inside any test `(... , async ({ page }) => {...})` block) — your editor will show you its exact type, which you can then import and reuse in your own function signatures.

SECTION 10 — Common Errors You'll Hit This Week, and Exactly How to Fix Them

10.1 “Property ‘x’ does not exist on type ‘y’ ”

When You'll See This Almost immediately, the first time you try to access a field on an object whose type TypeScript has inferred or been told, and that field genuinely isn't part of the declared shape.

What's Actually Happening TypeScript checked the object's known type (from an interface, or from what it inferred) and the field you're trying to reach isn't listed on it — either it's a genuine typo, or the type declaration is incomplete/wrong for what the object actually contains at runtime.

Example and Fix

```
interface Cohort { id: string; name: string; memberCount: number; }

function logCohort(cohort: Cohort) {
  console.log(cohort.membersCount); // ERROR — the interface says
    ↪ 'memberCount', not 'membersCount'
}
```

```
// Fix 1 – it was just a typo
function logCohort(cohort: Cohort) {
  console.log(cohort.memberCount); // matches the interface exactly
}

// Fix 2 – if the field genuinely should exist, add it to the interface
interface Cohort { id: string; name: string; memberCount: number;
  ↪ lastExportedAt?: string; }
```

Quick Practice Tip Before assuming your code is wrong, double-check the interface itself — sometimes the *type declaration* is the thing that’s outdated or incomplete, not your usage of it.

10.2 “Object is possibly ‘undefined’ ” or “possibly ‘null’ ”

When You’ll See This Constantly, once you enable TypeScript’s strict mode (which is Playwright’s and most Cypress starter configs’ default) — any time you access a property on something that *might* not exist.

What’s Actually Happening TypeScript is protecting you from a real, common runtime crash — accessing a property on undefined or null throws an error the moment your test actually runs. In strict mode, TypeScript refuses to let this possibility slide by unchecked.

Example and Fix

```
interface Cohort { name: string; lastExport?: { completedAt: string }; }

function getExportTime(cohort: Cohort): string {
  return cohort.lastExport.completedAt; // ERROR – lastExport might be
  ↪ undefined
}

// Fix – use optional chaining (Section 6.3) plus a fallback
function getExportTime(cohort: Cohort): string {
  return cohort.lastExport?.completedAt ?? 'Never exported';
}
```

Quick Practice Tip This error is almost always fixed with `?.` and `??`, covered in Section 6.3 — if you see this error a lot in your first week, it’s a strong sign you should go re-read that section closely, since it’ll come up constantly.

10.3 “Argument of type ‘X’ is not assignable to parameter of type ‘Y’ ”

When You’ll See This Whenever you call a function and pass something of the wrong type — extremely common while you’re still learning exactly what each function expects.

What’s Actually Happening Exactly what the message says — the function’s declared parameter type doesn’t match what you’re passing in. This is one of the clearest, most beginner-friendly TypeScript error messages, because it tells you both types directly.

Example and Fix

```
function seedCohort(size: number): void { /* ... */ }

seedCohort('250'); // ERROR – Argument of type 'string' is not assignable to
↳ parameter of type 'number'

// Fix – pass the actually-correct type
seedCohort(250);
```

Quick Practice Tip Read this error slowly, left to right — it always tells you “what you gave” (type X) and “what was expected” (type Y); the fix is almost always changing the value you’re passing, not the function’s signature, unless you genuinely intended the function to accept that type too.

10.4 A Custom Cypress Command Shows a Type Error Even Though It Works Fine When You Run It

When You’ll See This The very first time you add a new `Cypress.Commands.add(...)` and try to call it from a spec file.

What’s Actually Happening This is Section 9.2’s exact scenario — you wrote the command’s runtime logic, but haven’t yet told TypeScript about it via the `.d.ts` declaration file, so TypeScript has no idea the command exists at all.

Example and Fix

```
// You wrote this – command works perfectly when the test actually runs
Cypress.Commands.add('seedCohort', (size: number) => {
  return cy.request('POST', '/api/cohorts', { size }).its('body.id');
});

// But this shows a type error, because TypeScript doesn't know 'seedCohort'
↳ exists:
cy.seedCohort(100); // ERROR: Property 'seedCohort' does not exist on type
↳ 'Chainable'

// Fix – add the declaration
// cypress/support/index.d.ts
declare namespace Cypress {
  interface Chainable {
    seedCohort(size: number): Chainable<string>;
  }
}
```

Quick Practice Tip If a command “works but shows red” specifically, that’s the signature of a missing `.d.ts` entry — it’s almost never anything else.

10.5 “This expression is not callable” or “Cannot invoke an object which is possibly undefined”

When You’ll See This When calling a function that came from an optional property, or one that TypeScript thinks might not actually be a function at that point.

What's Actually Happening Similar root cause to 10.2 — TypeScript won't let you call something that it can't guarantee exists and is actually a function, at the specific point in your code where you're trying to call it.

Example and Fix

```
interface TestConfig { onComplete?: () => void; }

function runTest(config: TestConfig) {
  config.onComplete(); // ERROR — onComplete might be undefined, so it might
    ↪ not be callable
}

// Fix — check it exists first
function runTest(config: TestConfig) {
  config.onComplete?.(); // optional chaining works on function calls too —
    ↪ calls it only if it exists
}
```

Quick Practice Tip `?.()` (optional chaining directly before a function call) is the specific pattern for “call this function only if it actually exists” — it's the same `?.` from Section 6.3, just applied to a function call instead of a property access.

10.6 tsconfig strict Mode — What It Actually Turns On, in Plain English

When You'll See This Not an error itself, but the setting that determines how many of the errors above you'll see at all — worth understanding once, early.

What's Actually Happening `strict: true` in `tsconfig.json` (the default in Playwright's starter config, and recommended for Cypress) turns on a bundle of individual strictness checks at once — the most impactful for you day-to-day is `strictNullChecks`, which is specifically what makes undefined/null errors (10.2) show up at all. Without strict mode, TypeScript is far more permissive, and lets a lot of real bugs through silently — which defeats much of the reason to use TypeScript in the first place.

Example

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true, // turns on strictNullChecks, noImplicitAny, and several
      ↪ others together
    "target": "ES2021",
    "module": "commonjs"
  }
}
```

Quick Practice Tip Don't turn strict off to make errors go away faster — every error it surfaces is a real, specific thing TypeScript found questionable about your code; working through them (even slowly, while learning) is exactly how you build the instincts this whole handbook is trying to give you.

SECTION 11 — Loops, Arrays, Maps & Sets

11.1 The Three Loops You'll Actually Use: `for...of`, `for...in`, and Plain `for`

Level: Foundational | **You'll Use This:** Constantly

Why It Matters For Your Work Almost every test that touches a list of roles, cohort IDs, or table rows needs a loop — knowing which loop to reach for (and why the other two exist) stops you from writing the wrong one out of habit.

Plain-English Explanation `for...of` walks through the *values* in an array — this is the one you'll use nearly every time in test code. `for...in` walks through the *keys* of an object (rarely what you want for arrays — it gives you index numbers as strings, not the actual items, which trips people up). A plain `for` (`let i = 0; ...`) gives you full manual control over the index, useful when you genuinely need the index itself (like comparing item `i` to item `i + 1`).

Playwright Example

```
const roles: string[] = ['admin', 'analyst', 'auditor'];

// for...of — the one you'll reach for almost every time
for (const role of roles) {
  await page.goto(`/login?role=${role}`);
  await expect(page.getByText(`Logged in as ${role}`)).toBeVisible();
}

// Plain for — only when you actually need the index
for (let i = 0; i < roles.length; i++) {
  console.log(`Role ${i + 1} of ${roles.length}: ${roles[i]}`);
}
```

Cypress Example

```
const cohortNames: string[] = ['Cohort A', 'Cohort B', 'Cohort C'];

before(() => {
  for (const name of cohortNames) {
    cy.createTestCohort({ name });
  }
});
```

Common Mistake Using `for...in` on an array, expecting the actual values — `for (const i in roles)` gives you `'0'`, `'1'`, `'2'` (index numbers, as strings), not `'admin'`, `'analyst'`, `'auditor'`. This is a very common beginner trap coming from other languages; `for...in` is for objects, `for...of` is for arrays.

Quick Practice Tip If you're iterating an array and just want each item, default to `for...of` every time — you'll rarely need the other two in typical test code.

11.2 `.forEach()` — the Array Method Version of a Loop

Level: Core | **You'll Use This:** Frequently, especially in Cypress

Why It Matters For Your Work `.forEach()` does the same job as `for...of`, but as a method

call taking a callback function — you'll see it constantly in Cypress code specifically, since Cypress's command-queueing model works well with this style.

Plain-English Explanation `array.forEach(callback)` runs your callback function once for each item in the array, passing the item (and optionally its index) in. It looks different from a `for...of` loop but accomplishes the same thing — pick whichever reads more clearly for a given piece of code; many teams have a house style preference for one over the other.

Playwright Example

```
const cohortIds: string[] = ['c1', 'c2', 'c3'];

cohortIds.forEach((id, index) => {
  console.log(`Cohort ${index}: ${id}`);
});
```

Cypress Example

```
const cohortNames: string[] = ['Cohort A', 'Cohort B', 'Cohort C'];

before(() => {
  cohortNames.forEach((name) => {
    cy.createTestCohort({ name }); // very common pattern in Cypress seeding
    ↪ code
  });
});
```

Common Mistake Trying to use `await` inside a `.forEach()` callback in Playwright, expecting it to pause between iterations — `.forEach()` doesn't wait for each callback to finish before starting the next one, so all your `await`s inside it effectively race each other instead of running in order. Use a `for...of` loop instead whenever you need to `await` something on each iteration.

Quick Practice Tip Rule of thumb: if the loop body needs `await` (true for almost everything in Playwright), use `for...of`. If it's a simple, synchronous action per item (common in Cypress, since Cypress commands queue internally), `.forEach()` is fine.

11.3 `.map()` and `.filter()` — Transforming and Narrowing Arrays Without a Loop

Level: Core | **You'll Use This:** Constantly

Why It Matters For Your Work Instead of writing a loop that builds up a new array by hand, `.map()` and `.filter()` do that in one line — you'll see these two everywhere in real Playwright/Cypress codebases, especially when working with API response data.

Plain-English Explanation `.map()` takes an array and a callback, and returns a *new* array where each item has been transformed by that callback — same length as the original, different (or same) content. `.filter()` takes an array and a callback that returns `true/false`, and returns a *new* array containing only the items where the callback returned `true` — shorter or equal length, same content.

Playwright Example

```
interface Cohort { id: string; name: string; status: string; }
```

```
const cohorts: Cohort[] = await apiClient.listCohorts();

// map – extract just the names into a new array of strings
const cohortNames: string[] = cohorts.map((cohort) => cohort.name);

// filter – keep only the active ones
const activeCohorts: Cohort[] = cohorts.filter((cohort) => cohort.status ===
  ↪ 'active');

// they combine naturally
const activeCohortNames: string[] = cohorts
  .filter((cohort) => cohort.status === 'active')
  .map((cohort) => cohort.name);
```

Cypress Example

```
cy.get('[data-testid="cohort-row"]').then(($rows) => {
  const names: string[] = [...$rows].map((row) => row.textContent?.trim() ??
    ↪ '');
  const nonEmptyNames: string[] = names.filter((name) => name.length > 0);
  cy.log(`Found ${nonEmptyNames.length} named rows`);
});
```

Common Mistake Using `.map()` when you actually wanted `.filter()`, or vice versa — a quick way to remember: `.map()` always returns the *same number* of items (transformed), `.filter()` always returns the *same or fewer* items (kept, unchanged). If your output array is a different length than expected, check you’re using the right one.

Quick Practice Tip Read `.map()` as “turn each X into a Y” and `.filter()` as “keep only the Xs where this is true” — those two sentences cover the overwhelming majority of real uses you’ll write.

11.4 `.find()`, `.some()`, `.every()`, and `.includes()` — Asking Questions About an Array

Level: Core | **You’ll Use This:** Frequently

Why It Matters For Your Work Test assertions constantly need to ask “does this list contain X,” “is there an item matching Y,” or “do all items satisfy Z” — these four methods answer exactly those questions without writing a manual loop with a flag variable.

Plain-English Explanation `.find()` returns the *first item* matching a condition (or undefined if none match). `.some()` returns true/false for “does at least one item match.” `.every()` returns true/false for “do all items match.” `.includes()` is the simplest — true/false for “is this exact value present in the array.”

Playwright Example

```
interface Cohort { id: string; name: string; memberCount: number; }
const cohorts: Cohort[] = await apiClient.listCohorts();

const largeCohort = cohorts.find((c) => c.memberCount > 500); // first match,
  ↪ or undefined
const hasAnyLargeCohort: boolean = cohorts.some((c) => c.memberCount > 500);
```

```
const allCohortsHaveMembers: boolean = cohorts.every((c) => c.memberCount > 0);

const roles: string[] = ['admin', 'analyst', 'auditor'];
const isValidRole: boolean = roles.includes('admin'); // true
```

Cypress Example

```
cy.get('[data-testid="cohort-row"]').then(($rows) => {
  const names = [...$rows].map((row) => row.textContent);
  expect(names.includes('Diabetes Cohort')).to.be.true;
});
```

Common Mistake Using `.filter(...)[0]` to get “the first matching item” instead of `.find(...)` — `.filter()` builds an entire new array just to throw away everything except the first item, which is both less efficient and less clear about your actual intent than `.find()`.

Quick Practice Tip If your goal is “give me one specific item,” use `.find()`. If your goal is “give me a yes/no answer,” use `.some()`, `.every()`, or `.includes()` depending on exactly what you’re asking. Reaching for `.filter()` for either of these is almost always the wrong tool.

11.5 Map — a Better “Object as a Lookup Table”

Level: Core | **You’ll Use This:** Occasionally, very useful for grouping test results

Why It Matters For Your Work You’ve already used plain objects as lookup tables (Record<Role, string> in Section 8.3). A Map is a more powerful alternative — especially useful when your keys aren’t known in advance, like grouping API results by a dynamic field.

Plain-English Explanation A Map stores key-value pairs, similar to a plain object, but with a few real advantages for test code: keys can be *any* type (not just strings), it has a built-in `.size` property (no need for `Object.keys(obj).length`), and it preserves insertion order reliably. You create one with `new Map()`, add entries with `.set(key, value)`, and read with `.get(key)`.

Playwright Example

```
interface Cohort { id: string; status: string; memberCount: number; }
const cohorts: Cohort[] = await apiClient.listCohorts();

// Group cohorts by status — status values aren't known in advance
const cohortsByStatus = new Map<string, Cohort[]>();

for (const cohort of cohorts) {
  const existing = cohortsByStatus.get(cohort.status) ?? [];
  existing.push(cohort);
  cohortsByStatus.set(cohort.status, existing);
}

console.log(cohortsByStatus.get('active')?.length); // how many active cohorts
console.log(cohortsByStatus.size); // how many distinct statuses appeared
```

Cypress Example

```
// Typing a Map explicitly for a role-to-token cache during a test run
const tokenCache = new Map<string, string>();

Cypress.Commands.add('loginAs', (role: string) => {
  if (tokenCache.has(role)) {
    cy.log(`Reusing cached token for ${role}`);
  } else {
    cy.request('POST', '/api/auth/login', { role }).then((response) => {
      tokenCache.set(role, response.body.token);
    });
  }
});
```

Common Mistake Reaching for Map by default even when a plain object or Record<...> would be simpler and more idiomatic — for a small, fixed set of known string keys (like roles), a plain typed object is usually clearer; Map earns its place specifically when keys are dynamic, non-string, or when you need .size/insertion-order guarantees.

Quick Practice Tip Default to a plain object (or Record<>) first; reach for Map specifically when your keys are only known at runtime (not a fixed set you could write as a union type).

11.6 Set — a List With No Duplicates

Level: Core | **You'll Use This:** Occasionally, great for dedup and comparison checks

Why It Matters For Your Work A very common test-writing need is “make sure this list has no duplicates” or “get only the unique values from this array” — Set does both instantly, far more cleanly than manually looping and checking.

Plain-English Explanation A Set is a collection that automatically enforces uniqueness — adding the same value twice has no effect the second time. Creating one from an existing array (new Set(array)) is the single fastest way to deduplicate that array's values.

Playwright Example

```
const cohortStatuses: string[] = ['active', 'active', 'archived', 'active',
  ↪ 'pending'];

const uniqueStatuses = new Set(cohortStatuses);
console.log(uniqueStatuses.size); // 3 - 'active', 'archived', 'pending'

// Converting back to an array when you need array methods again
const uniqueStatusArray: string[] = [...uniqueStatuses];

// A real assertion use: confirm a table has no duplicate cohort IDs
const rowIds: string[] = await page.getByTestId('cohort-row').evaluateAll(
  (rows) => rows.map((r) => r.getAttribute('data-cohort-id') ?? '' )
);
expect(new Set(rowIds).size).toBe(rowIds.length); // true only if there are
  ↪ zero duplicates
```

Cypress Example

```
cy.get('[data-testid="cohort-row"]').then(($rows) => {  
  const ids = [...$rows].map((row) => row.getAttribute('data-cohort-id') ??  
    ↪ '');  
  const uniqueIds = new Set(ids);  
  expect(uniqueIds.size).to.eq(ids.length); // fails if any duplicate row IDs  
    ↪ exist  
});
```

Common Mistake Forgetting that a Set isn't an array — you can't use `.map()` or `.filter()` directly on it. Convert it back to an array first with the spread syntax (`[...mySet]`) whenever you need array methods.

Quick Practice Tip Whenever you catch yourself writing “check this array has no duplicates” or “get only the unique values,” immediately reach for `new Set(array)` — comparing `set.size` to `array.length` is the whole duplicate-check in one line.
